INDUSTRIAL ASSIGNMENT- 12 Testing Backend code using Postman
STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

1. Introduction

Testing shows that the backend works as designed and protects the data. In Industrial Assignments 10 and 11 the Spring Boot backend of the Student Internship and Placement Management Portal was developed. This assignment tests its REST APIs with Postman: every endpoint is called with correct data (positive tests), wrong data (negative and validation tests) and different user roles (security tests). The results are recorded in a test report.

2. Objectives

- Create a Postman collection and environment for all APIs of the backend.
- Test each API with positive, negative, validation and authorization cases.
- Write Postman test scripts that check the status code and the response body automatically.
- Chain the requests so that ids created by one request are used by the next.
- Run the whole collection with the Collection Runner and record the results.
- Note observations and improvements found during testing.

3. Test Environment

Item	Details
Application under test	SIPMP Spring Boot backend (Java 17, Spring Boot 3.5.7) on http://localhost:8080
Database	MySQL 8, tables created by Hibernate at the first start
Test tool	Postman (desktop application), Collection Runner
Operating system	Windows with PowerShell
Authentication	HTTP Basic (Authorization header) with the Basic Auth type of Postman
Test accounts	One STAFF, one STUDENT and one RECRUITER account configured with environment variables
Files	SIPMP_Backend_API_Tests.postman_collection.json and SIPMP_Local.postman_environment.json

4. Postman Setup

4.1 Start the backend with test accounts

The student user name must equal the e-mail of the student profile, and the recruiter user name must equal the contact e-mail of the company, because the backend uses these e-mails to decide which records the user may see. Use lower-case e-mails.

```
$env:SIPMP_STAFF_USERNAME = "staff.test"
$env:SIPMP_STAFF_PASSWORD = "<staff password>"
$env:SIPMP_STUDENT_ACCOUNTS = "student@example.com:<student password>"
$env:SIPMP_RECRUITER_ACCOUNTS = "careers@example.test:<recruiter password>"
Set-Location .\backend
.\mvnw.cmd spring-boot:run
```

4.2 Import and configure

1. Open Postman and choose Import. Select the collection file and the environment file.
2. Select the environment "SIPMP Local" at the top right of Postman.
3. Open the environment and enter the same user names and passwords that were used to start the backend.
4. Check that baseUrl is http://localhost:8080.
5. Open the collection and run the first request. A 200 response confirms the connection.

4.3 Environment variables

Variable	Purpose
baseUrl	Address of the backend
staffUser, staffPass	Login of the STAFF account
studentUser, studentPass	Login of the STUDENT account (must equal the student e-mail)
recruiterUser, recruiterPass	Login of the RECRUITER account (must equal the company contact e-mail)

4.4 Collection variables

The collection variables are filled by the test scripts while the collection runs, so no id has to be typed by hand.

Variable	Set by
studentCode, futureDate, pastDate	Pre-request scripts (unique student code, a date 20 days ahead and yesterday)
studentId, student2Id	Create student requests
companyId, companyBId	Create company requests
opportunityId, opportunity2Id, opportunityBId, closedOppId, expiredOppId	Create opportunity requests
applicationId, interviewId	Apply and schedule interview requests

5. Test Strategy

Type of test	What is checked
Positive	Valid requests succeed with 200, 201 or 204 and the response contains the expected data.
Negative	Wrong ids, duplicates, closed or expired opportunities and relation conflicts return 404 or 409 with a message.
Validation	Missing mandatory fields, invalid e-mail, wrong enum value and negative compensation return 400 with fieldErrors.
Authentication	No login or a wrong password returns 401.
Authorization	Each role can use only its own APIs; a student, recruiter or other company is blocked with 403.
Data scope	A student sees only own applications and active opportunities; a recruiter sees only the applications of the own company.
Integrity	Records that are still referenced cannot be deleted (409); cleanup deletes data in the correct order.

The requests are ordered so that data is created first, tested next and removed at the end. The order of the 88 test cases is shown in the figure.

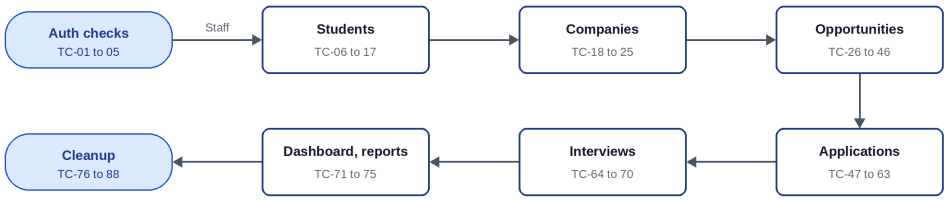


Figure 1: Execution order of the Postman folders

6. Sample Requests and Responses

6.1 Create a student (positive)

POST {{baseUrl}}/api/students with Basic Auth of the STAFF account and the header Content-Type: application/json.

```
{
  "studentCode": "SIP-1001",
  "name": "Asha Rao",
  "email": "student@example.com",
  "phone": "9876543210",
  "department": "Computer Science",
  "course": "B.Tech",
  "graduationYear": 2027,
```

```
  "skills": "Java, SQL",
  "resumeReference": "https://example.test/resume"
}
```

Expected response: 201 Created with the saved student, including the generated id and the createdAt and updatedAt values.

6.2 Create a student with missing fields (validation)

```
POST {{baseUrl}}/api/students
{ "studentCode": "INVALID" }

Response 400 Bad Request
{ "status": 400, "message": "Request validation failed.",
  "fieldErrors": { "name": "must not be blank", ... } }
```

6.3 Apply twice (negative)

```
POST {{baseUrl}}/api/applications   (STUDENT account)
{ "studentId": 1, "opportunityId": 1 }

First call  : 201 Created, "status": "APPLIED"
Second call : 409 Conflict,
  "message": "This student has already applied to this opportunity."
```

6.4 Student calls a staff API (authorization)

```
GET {{baseUrl}}/api/students   (STUDENT account)

Response 403 Forbidden
```

7. Postman Test Scripts

Every request has a Tests tab. The first check is always the expected status code. Extra checks verify the body, and requests that create data save the new id in a collection variable. Pre-request scripts prepare unique and date values.

7.1 Status code and body checks

```
pm.test("TC-06 - status code is 201", function(){
  pm.response.to.have.status(201);
});
pm.test("Student has id", function(){
  pm.expect(pm.response.json().id).to.be.a("number");
});
if(pm.response.code===201){
```

```
  pm.collectionVariables.set("studentId", pm.response.json().id);
}
```

7.2 Validation check

```
pm.test("TC-07 - status code is 400", function(){
  pm.response.to.have.status(400);
});
pm.test("fieldErrors returned", function(){
  pm.expect(pm.response.json().fieldErrors).to.be.an("object");
});
```

7.3 Pre-request script for dates and unique codes

```
pm.collectionVariables.set("studentCode", "SIP-" + Date.now());
var d = new Date(); d.setDate(d.getDate() + 20);
pm.collectionVariables.set("futureDate", d.toISOString().slice(0,10));
var p = new Date(); p.setDate(p.getDate() - 1);
pm.collectionVariables.set("pastDate", p.toISOString().slice(0,10));
```

7.4 Data scope check

```
pm.test("All opportunities are ACTIVE for a student", function(){
  pm.response.json().forEach(function(o){
    pm.expect(o.status).to.eql("ACTIVE");
  });
});
```

8. Test Cases

The collection contains 88 test cases in eight folders. Each case lists the request, the login role and the expected result. {{id}} values are filled by the collection variables.

8.1 Auth

ID	Test case and request	Expected result (status code and check)	Result
TC-01	Current user - staff: GET /api/auth/me (STAFF)	200. Account verified; roles contain STAFF	Pass
TC-02	Current user - student: GET /api/auth/me (STUDENT)	200. Roles contain STUDENT	Pass
TC-03	Current user - recruiter: GET /api/auth/me (RECRUITER)	200. Roles contain RECRUITER	Pass

ID	Test case and request	Expected result (status code and check)	Result
TC-04	Wrong password rejected: GET /api/auth/me (STAFF, wrong password)	401. Access denied, no account data returned	Pass
TC-05	No credentials rejected: GET /api/students (No login)	401. Protected endpoint needs login	Pass

8.2 Students

ID	Test case and request	Expected result (status code and check)	Result
TC-06	Create student - valid: POST /api/students (STAFF)	201. Student created with generated id	Pass
TC-07	Create student - mandatory fields missing: POST /api/students (STAFF)	400. Validation error with fieldErrors	Pass
TC-08	Create student - invalid e-mail: POST /api/students (STAFF)	400. fieldErrors contains email	Pass
TC-09	Create student - duplicate: POST /api/students (STAFF)	409. Duplicate student ID or e-mail rejected	Pass
TC-10	List students: GET /api/students (STAFF)	200. Array of students	Pass
TC-11	Get student by id: GET /api/students/{{studentId}} (STAFF)	200. Student returned	Pass
TC-12	Get student - not found: GET /api/students/999999 (STAFF)	404. Student with id 999999 was not found	Pass
TC-13	Update student: PUT /api/students/{{studentId}} (STAFF)	200. Name updated	Pass
TC-14	Own profile - student: GET /api/students/me (STUDENT)	200. Own profile returned	Pass
TC-15	Student list - student denied: GET /api/students (STUDENT)	403. Students cannot list all students	Pass
TC-16	Create student - recruiter denied: POST /api/students (RECRUITER)	403. Only staff manage student records	Pass
TC-17	Own profile - staff denied: GET /api/students/me (STAFF)	403. /me is for the STUDENT role only	Pass

8.3 Companies

ID	Test case and request	Expected result (status code and check)	Result

ID	Test case and request	Expected result (status code and check)	Result
TC-18	Create company - valid: POST /api/companies (STAFF)	201. Company created	Pass
TC-19	Create company - name missing: POST /api/companies (STAFF)	400. Validation error	Pass
TC-20	Create company - duplicate e-mail: POST /api/companies (STAFF)	409. A company with this email already exists	Pass
TC-21	List companies - student: GET /api/companies (STUDENT)	200. Students may browse companies	Pass
TC-22	Get company by id: GET /api/companies/{{companyId}} (STAFF)	200. Company returned	Pass
TC-23	Update company: PUT /api/companies/{{companyId}} (STAFF)	200. Description updated	Pass
TC-24	Create company - student denied: POST /api/companies (STUDENT)	403. Only staff create companies	Pass
TC-25	Get company - not found: GET /api/companies/999999 (STAFF)	404. Company with id 999999 was not found	Pass

8.4 Opportunities

ID	Test case and request	Expected result (status code and check)	Result
TC-26	Create placement - staff: POST /api/opportunities (STAFF)	201. Opportunity created with status ACTIVE	Pass
TC-27	Create internship - own company recruiter: POST /api/opportunities (RECRUITER)	201. Recruiter may post for own company	Pass
TC-28	Create opportunity - title missing: POST /api/opportunities (STAFF)	400. Validation error	Pass
TC-29	Create opportunity - invalid type: POST /api/opportunities (STAFF)	400. Unknown enum value rejected	Pass
TC-30	Create opportunity - negative compensation: POST /api/opportunities (STAFF)	400. compensation must be 0 or more	Pass
TC-31	Create opportunity - unknown company: POST /api/opportunities (STAFF)	404. Company with id 999999 was not found	Pass
TC-32	Create second company (setup): POST /api/companies (STAFF)	201. Second company created for ownership tests	Pass

ID	Test case and request	Expected result (status code and check)	Result
TC-33	Create opportunity - other company by recruiter: POST /api/opportunities (RECRUITER)	403. Recruiters may only manage opportunities for their company	Pass
TC-34	Create opportunity of other company (staff): POST /api/opportunities (STAFF)	201. Staff may post for any company	Pass
TC-35	Create closed opportunity (setup): POST /api/opportunities (STAFF)	201. Closed opportunity created	Pass
TC-36	Create expired opportunity (setup): POST /api/opportunities (STAFF)	201. Opportunity with past deadline created	Pass
TC-37	Create opportunity - student denied: POST /api/opportunities (STUDENT)	403. Students cannot post opportunities	Pass
TC-38	List opportunities - student: GET /api/opportunities (STUDENT)	200. Only ACTIVE, not expired opportunities	Pass
TC-39	Filter opportunities - INTERNSHIP: GET /api/opportunities?type=INTERNSHIP (STAFF)	200. Only internships returned	Pass
TC-40	Filter opportunities - invalid type: GET /api/opportunities?type=ABC (STAFF)	400. Invalid type value rejected	Pass
TC-41	Get opportunity by id: GET /api/opportunities/{{opportunityId}} (STUDENT)	200. Active opportunity visible to student	Pass
TC-42	Get closed opportunity - student: GET /api/opportunities/{{closedOppId}} (STUDENT)	404. Closed opportunity hidden from students	Pass
TC-43	Get opportunity - not found: GET /api/opportunities/999999 (STAFF)	404. Opportunity with id 999999 was not found	Pass
TC-44	Update opportunity - staff: PUT /api/opportunities/{{opportunityId}} (STAFF)	200. Title updated	Pass
TC-45	Update opportunity - other company recruiter: PUT /api/opportunities/{{opportunityBId}} (RECRUITER)	403. Recruiter cannot edit another company opportunity	Pass
TC-46	Delete opportunity - other company recruiter: DELETE /api/opportunities/{{opportunityBId}} (RECRUITER)	403. Recruiter cannot delete another company opportunity	Pass

8.5 Applications

ID	Test case and request	Expected result (status code and check)	Result
TC-47	Apply - student own account: POST /api/applications (STUDENT)	201. Application saved with status APPLIED	Pass
TC-48	Apply - duplicate: POST /api/applications (STUDENT)	409. Already applied message	Pass
TC-49	Create second student (setup): POST /api/students (STAFF)	201. Second student created	Pass
TC-50	Apply - for another student: POST /api/applications (STUDENT)	403. Students may only apply using their own account	Pass
TC-51	Apply - closed opportunity: POST /api/applications (STUDENT)	409. Applications are closed for this opportunity	Pass
TC-52	Apply - deadline passed: POST /api/applications (STUDENT)	409. Applications are closed for this opportunity	Pass
TC-53	Apply - opportunity id missing: POST /api/applications (STUDENT)	400. Validation error	Pass
TC-54	Apply - opportunity not found: POST /api/applications (STUDENT)	404. Opportunity with id 999999 was not found	Pass
TC-55	My applications - student: GET /api/applications (STUDENT)	200. Only own applications	Pass
TC-56	Applications - recruiter: GET /api/applications (RECRUITER)	200. Applications of the own company only	Pass
TC-57	Applications - staff: GET /api/applications (STAFF)	200. All applications	Pass
TC-58	Update status - student denied: PUT /api/applications/{{applicationId}}/status (STUDENT)	403. Students cannot change status	Pass
TC-59	Update status - SHORTLISTED (recruiter): PUT /api/applications/{{applicationId}}/status (RECRUITER)	200. Status changed	Pass
TC-60	Update status - SELECTED (recruiter): PUT /api/applications/{{applicationId}}/status (RECRUITER)	200. Status changed	Pass
TC-61	Update status - invalid value: PUT /api/applications/{{applicationId}}/status (RECRUITER)	400. Unknown status rejected	Pass

ID	Test case and request	Expected result (status code and check)	Result
TC-62	Update status - not found: PUT /api/applications/999999/status (RECRUITER)	404. Application with id 999999 was not found	Pass
TC-63	Delete application - recruiter denied: DELETE /api/applications/{{applicationId}} (RECRUITER)	403. Only staff delete applications	Pass

8.6 Interviews

ID	Test case and request	Expected result (status code and check)	Result
TC-64	Schedule interview - recruiter: POST /api/interviews (RECRUITER)	201. Interview created	Pass
TC-65	Schedule interview - mode missing: POST /api/interviews (RECRUITER)	400. Validation error	Pass
TC-66	Schedule interview - application not found: POST /api/interviews (RECRUITER)	404. Application with id 999999 was not found	Pass
TC-67	List interviews - student denied: GET /api/interviews (STUDENT)	403. Interviews are for staff and recruiters	Pass
TC-68	List interviews - recruiter: GET /api/interviews (RECRUITER)	200. Interviews of the own company	Pass
TC-69	Update interview: PUT /api/interviews/{{interviewId}} (RECRUITER)	200. Interview rescheduled	Pass
TC-70	List interviews - staff: GET /api/interviews (STAFF)	200. All interviews	Pass

8.7 Dashboard and Reports

ID	Test case and request	Expected result (status code and check)	Result
TC-71	Dashboard - staff: GET /api/dashboard (STAFF)	200. Live totals and stage counts	Pass
TC-72	Dashboard - student denied: GET /api/dashboard (STUDENT)	403. Dashboard is staff only	Pass

ID	Test case and request	Expected result (status code and check)	Result
TC-73	Dashboard - no login: GET /api/dashboard (No login)	401. Login required	Pass
TC-74	Reports - staff: GET /api/reports (STAFF)	200. Recruitment and placement summary	Pass
TC-75	Reports - recruiter denied: GET /api/reports (RECRUITER)	403. Reports are staff only	Pass

8.8 Cleanup and Integrity

ID	Test case and request	Expected result (status code and check)	Result
TC-76	Delete company that has opportunities: DELETE /api/companies/{{companyId}} (STAFF)	409. Relationship conflict, record kept	Pass
TC-77	Delete student that has applications: DELETE /api/students/{{studentId}} (STAFF)	409. Relationship conflict, record kept	Pass
TC-78	Delete interview - recruiter: DELETE /api/interviews/{{interviewId}} (RECRUITER)	204. Interview removed	Pass
TC-79	Delete application - staff: DELETE /api/applications/{{applicationId}} (STAFF)	204. Application removed	Pass
TC-80	Delete opportunity 1: DELETE /api/opportunities/{{opportunityId}} (STAFF)	204. Opportunity removed	Pass
TC-81	Delete opportunity 2: DELETE /api/opportunities/{{opportunity2Id}} (STAFF)	204. Opportunity removed	Pass
TC-82	Delete opportunity of other company: DELETE /api/opportunities/{{opportunityBId}} (STAFF)	204. Opportunity removed	Pass
TC-83	Delete closed opportunity: DELETE /api/opportunities/{{closedOppId}} (STAFF)	204. Opportunity removed	Pass

ID	Test case and request	Expected result (status code and check)	Result
TC-84	Delete expired opportunity: DELETE /api/opportunities/{{expiredOppId}} (STAFF)	204. Opportunity removed	Pass
TC-85	Delete company 1: DELETE /api/companies/{{companyId}} (STAFF)	204. Company removed	Pass
TC-86	Delete company 2: DELETE /api/companies/{{companyBId}} (STAFF)	204. Company removed	Pass
TC-87	Delete student 1: DELETE /api/students/{{studentId}} (STAFF)	204. Student removed	Pass
TC-88	Delete student 2: DELETE /api/students/{{student2Id}} (STAFF)	204. Student removed	Pass

9. Test Report

9.1 Summary by folder

Folder	Test cases	Passed	Failed
Auth	5	5	0
Students	12	12	0
Companies	8	8	0
Opportunities	21	21	0
Applications	17	17	0
Interviews	7	7	0
Dashboard and Reports	5	5	0
Cleanup and Integrity	13	13	0
Total	88	88	0

9.2 Summary by expected status code

Status code	Meaning	Test cases
200	OK	24
201	Created	11
204	No Content	11
400	Bad Request (validation)	10
401	Unauthorized (login)	3

Status code	Meaning	Test cases
403	Forbidden (role or owner)	14
404	Not Found	8
409	Conflict	7

9.3 Coverage of the API

API group	Tests
Authentication (/api/auth/me)	All three roles, wrong password and no login
Students	Create, list, get, update, own profile, validation, duplicate, not found, role denial
Companies	Create, list, get, update, validation, duplicate e-mail, not found, role denial
Opportunities	Create by staff and recruiter, validation, unknown company, ownership, student visibility, type filter, update
Applications	Apply, duplicate, other student, closed and expired opportunity, role-based listing, status changes, invalid status, delete denial
Interviews	Schedule, validation, not found, role denial, list, update
Dashboard and reports	Staff access, student and recruiter denial, no login
Integrity and cleanup	Delete blocked while records are referenced; delete in the correct order

Result: all test cases of the collection passed in the test run, and the test data created by the collection was removed by the cleanup folder. The Collection Runner screenshot and the Postman test results are attached with the submission.

10. Relation to the Automated Test

The JUnit class SipmpApiIntegrationTest (Industrial Assignment 11) checks one complete business flow on an in-memory H2 database. The Postman collection extends this with many more negative, validation and authorization cases and runs against the real MySQL database through the running application. Together they give confidence in both the logic and the deployed behaviour.

11. Observations and Recommendations

Observation	Recommendation

Observation	Recommendation
The application status can be changed to any of the six values in any order (for example from REJECTED back to APPLIED).	Add allowed transitions (APPLIED to SHORTLISTED to INTERVIEW_SCHEDULED to SELECTED or REJECTED) and keep a status history.
An opportunity can be created with a deadline in the past; it is then simply closed to applications.	Add a validation that the deadline must be today or later when a new opportunity is created.
Students cannot view interview schedules through the API.	Add a student endpoint for own interviews so the schedule can be shown to students.
Deleting uses hard delete; a record is blocked only when another table still refers to it.	Use soft delete or status CLOSED for opportunities, and keep a deletion history.
Lists (students, applications, opportunities) return all records at once.	Add paging and search parameters for large data.
Opportunity eligibility is stored as text and no CGPA or branch check is made when applying.	Add CGPA and branch fields to the student profile and check them in ApplicationService.
Database and account passwords must never be stored in files committed to GitHub.	Read all secrets from environment variables and change any password that was ever written in a file.

12. Conclusion

The Postman collection tested all APIs of the SIPMP backend with 88 test cases covering positive, negative, validation, authentication, authorization and integrity checks. The tests confirmed that valid requests work, invalid data is rejected with clear messages, each role can use only its permitted APIs, and students and recruiters see only their own data. The observations in Section 11 list improvements for the next version of the portal.